Mohamed Ibrahim Mohamed

Frontend Engineer (React / Next.js / TypeScript)

Location: Mansoura, Egypt | Timezone: UTC+2
Available for full-time remote work

Phone: +20 155 065 4064 | Email: mohamedelboraei138@gmail.com
LinkedIn | GitHub

PROFESSIONAL SUMMARY

Frontend Engineer specializing in React and Next.js, focused on building secure, scalable, production-grade web applications. Experienced in shipping full-stack systems with strong emphasis on server-side security, normalized state management, and feature-based architecture. Proven track record of delivering measurable outcomes in performance, security, and scalability, including edge-level authentication blocking unauthorized access before render, 100% migration of database writes to Server Actions, and real-time analytics dashboards in live environments.

TECHNICAL SKILLS

Core: React 19, Next.js 15, TypeScript, JavaScript (ES2023)
State Management: Redux Toolkit, Zustand, React Query, Context API
Forms & Validation: React Hook Form, Zod
Styling: Tailwind CSS, Material UI (MUI), Framer Motion
Backend / BaaS: Supabase (PostgreSQL, Auth, Storage), REST APIs, Server Actions
Tooling: Vite, Git, Conventional Commits, Vercel, n8n Webhooks

---

PROJECTS

AI Document Assistant | Full-Stack Developer

Live: https://ai-document-assistant-beryl.vercel.app/
GitHub: https://github.com/mohamed0155065/Ai-document-assistant-

- Architected and developed a multi-tenant, project-based AI document processing platform enabling users to create isolated workspaces, upload multiple files, and perform AI-driven document analysis through custom prompt engineering.

- Engineered robust document ingestion and parsing pipelines supporting PDF, DOCX, and TXT formats using pdf2json and mammoth, transforming raw files into normalized, structured text optimized for downstream LLM processing.
- Designed and implemented a scalable multi-document context aggregation layer that merges extracted content with user-defined prompts to generate coherent AI responses using Groq API for low-latency LLM inference.
- Built a modular and reusable frontend architecture using Next.js and TypeScript, featuring drag-and-drop file uploads, document lifecycle management, analysis history tracking, and protected dashboard workflows.
- Integrated Supabase for authentication, PostgreSQL database management, and cloud storage, ensuring secure user isolation, persistent project data, and structured storage of AI-generated outputs.
- Applied clean architecture principles with strict separation of concerns across ingestion pipelines, AI orchestration layer, storage management, and UI components to ensure scalability, maintainability, and extensibility.

Stack: Next.js, TypeScript, Tailwind CSS, Supabase (Auth, DB, Storage), Groq API, pdf2json, mammoth

---

Marketly — Production-Grade E-Commerce Platform (2026)

Live: https://e-commerce-v7pt.vercel.app
GitHub: https://github.com/mohamed0155065/e-commerce

• Architected edge-level authentication using Next.js Middleware + Supabase, blocking unauthorized admin access before page render — zero client-side exposure.
• Eliminated 100% of client-side data mutations by migrating all DB writes to Server Actions, reducing bundle size by ~25% and preventing data leakage.
• Built fully shareable, URL-driven search with debounced queries, cutting redundant DB calls by ~40% during typing.
• Delivered real-time sales analytics dashboard, transforming raw PostgreSQL order data into actionable revenue insights via Recharts.
• Resolved SSR/CSR hydration mismatches by implementing mounting guards on persisted Zustand state, ensuring stable rendering across sessions.

Stack: Next.js 15, TypeScript, Supabase, Zustand, Zod, Tailwind CSS, Recharts

---

Finance Dashboard | Full-Stack Developer Next.js | TypeScript

Live : https://beamish-concha-1297ed.netlify.app/

GitHub : https://github.com/mohamed0155065/finance-dashboard

- Built a responsive financial dashboard with real-time data visualization and role-based access control using Next.js and TypeScript.
- Implemented interactive Line and Pie charts displaying balance trends and spending patterns with 100% type-safe code via Recharts.
- Designed dual-role UI (Admin/Viewer) with state management via Context API, enabling secure transaction CRUD operations.
- Achieved fully responsive design with dark mode support using Tailwind CSS v4, optimized for mobile, tablet, and desktop.
- Deployed on Netlify with CI/CD pipeline, demonstrating production-ready code with clean architecture and reusable components.

Stack: Next.js, TypeScript, Tailwind CSS v4, Recharts, Context API, Lucide Icons

---

React Admin Dashboard (2025)

Live: https://react-admin-dashboard-bqcs.vercel.app
GitHub: https://github.com/mohamed0155065/React-admin-dashboard

• Built centralized API abstraction layer with unified error handling, eliminating duplicated API logic across 12+ components.
• Integrated n8n automation webhooks and diagnosed production routing failures that were blocking webhook delivery.
• Implemented schema-driven form validation with React Hook Form + Zod, achieving zero runtime type errors on submission.

Stack: React 19, TypeScript, React Hook Form, Zod, n8n, Tailwind CSS

Movies Management Application (2025)

Live: https://movies-app-sepia-nu.vercel.app
GitHub: https://github.com/mohamed0155065/Movies-app

• Engineered normalized Redux state using createEntityAdapter with memoized selectors, eliminating redundant re-renders during filtering and sorting.
• Reduced initial load time by ~30% via route-level code splitting using React.lazy + Suspense.
• Persisted user preferences via centralized global store, eliminating prop drilling across the component tree.

Stack: React, Redux Toolkit, TypeScript, Vite

Smart Task Manager (2025)

Live: https://smart-task-manager-olive-ten.vercel.app
GitHub: https://github.com/mohamed0155065/smart-task-manager

• Built modular MUI component system with dynamic theming (dark/light), real-time progress tracking, and priority-based filtering — shipped with zero layout-breaking regressions across multiple theme variants.
• Decoupled UI presentation from business logic, enabling isolated testing and reducing feature implementation time by ~25%.
• Architected centralized Zustand store for deeply nested components, eliminating all prop drilling from the state layer.

Stack: React, TypeScript, Material UI

LANGUAGES & AVAILABILITY

Arabic: Native
English: Professional (written & spoken)
Timezone: UTC+2 — Flexible overlap with EU/US teams